

Lab 9: Knative Traffic Management Experiment Report

1. Experiment Structure Requirements

1.1 Experiment Name

Lab 9: Knative Traffic Management Experiment

1.2 Experiment Objectives

- Master the basic operations and core concepts of Knative Serving
- Understand the dual modes of Serverless applications (resident mode and auto-scaling-to-zero mode)
- Master the basic principles of Knative Eventing event-driven architecture

1.3 Experiment Principles

This section briefly explains the four core components of Knative (Serving, Eventing, Functions, Client) and their collaborative workflow, with emphasis on the request-based auto-scaling mechanism (including scaling down to zero) and the Broker/Trigger-based event processing model.

Knative Serving is responsible for deploying and managing Serverless applications, providing capabilities such as auto-scaling, traffic routing, and version management. Eventing enables event-driven architectures by providing a unified event delivery mechanism. Functions simplifies the development and deployment of Serverless functions. The Client (including kubectl and kn CLI) provides command-line tools for interacting with Knative components.

The collaborative workflow involves developers using the Client to deploy applications to Serving, which automatically scales the application based on request volume. For event-driven scenarios, Eventing connects event sources to applications through Broker and Trigger, enabling asynchronous event processing.

The request-based auto-scaling mechanism of Knative Serving uses metrics such as request count per second (RPS) to adjust the number of pod replicas dynamically. When there are no requests, it can scale down to zero replicas to save resources; when requests increase, it quickly scales up to meet the load demand. The Broker/Trigger-based event processing model uses a Broker as an event hub to receive events from various sources, and Triggers filter events and route them to

corresponding event consumers for processing.

1.4 Experiment Environment

- Kubernetes Cluster (Minikube)
- Knative Serving & Eventing
- kubectl & kn CLI

1.5 Experiment Content and Steps

This section clearly describes the step-by-step operations, divided into Knative Serving and Knative Eventing parts for detailed presentation.

1.5.1 Knative Serving Operations

1. **Create Hello World Service** Use the kn CLI to create a Hello World service with the following command: `kn service create hello-world --image gcr.io/knative-samples/helloworld-go --env TARGET="Knative"`
2. Verify the service creation using `kn service list` to check if the service status is "Ready".
3. **Verify Auto-Scaling** Obtain the service URL using `kn service describe hello-world -o url`, record the URL for subsequent access.
4. Use the `watch kubectl get pods` command to monitor the number of pod replicas in real time.
5. Simulate high concurrency requests using the `hey -z 1m <service-url>` command (ensure the hey tool is installed in advance).
6. Observe the increase in the number of pod replicas during the request period, and check if the replicas scale down to zero after the request stops (wait for about 60 seconds, the default cool-down period).
7. **Test Resident Mode** Modify the Hello World service to enable resident mode by setting the minScale parameter to 1: `kn service update hello-world --scale-min 1`
8. Use `kubectl get pods` to verify that there is always 1 pod running even without requests.

1.5.2 Knative Eventing Operations

9. **Create Broker** Create a default Broker in the current namespace using the command: `kn broker create default`
10. Verify the Broker status with `kn broker list`, ensuring the status is "Ready".
11. **Create Event Display Service** Deploy an event display service to receive and print events: `kn service create event-display --image`

`gcr.io/knative-samples/event-display`

12. Check the service status with `kn service list event-display`.
13. **Create Event Source (PingSource)** Create a PingSource that sends an event every 30 seconds: `kn source ping create ping-source --schedule "*/30 * * * *" --data '{"message": "Hello from PingSource"}' --sink broker:default`
14. Verify the PingSource status with `kn source ping list ping-source`.
15. **Create Trigger** Create a Trigger to route events from the Broker to the event-display service: `kn trigger create event-trigger --broker default --filter type=dev.knative.sources.ping --sink ksvc:event-display`
16. The filter parameter specifies that only events of type "dev.knative.sources.ping" are routed.
17. Check the Trigger status with `kn trigger list event-trigger`.
18. **Verify Event Flow** View the logs of the event-display service to check if events are received: `kubectl logs -l serving.knative.dev/service=event-display -c user-container -f`
19. Observe if a log entry is generated every 30 seconds, containing the message from the PingSource.

1.6 Experiment Results and Analysis

1.6.1 Knative Serving Results

- **Hello World Service Creation:** After executing the service creation command, the output of `kn service list` showed that the hello-world service was in "Ready" status, and the service URL was successfully obtained (e.g., `http://hello-world.default.192.168.49.2.nip.io`). Accessing the URL through a browser or curl command returned "Hello Knative!", indicating the service was deployed successfully. The result is shown in Figure 1.
- **Auto-Scaling Verification:** During the high concurrency test with the hey tool, the number of pod replicas increased from 0 to 3 (the specific number depends on the request pressure) within 10 seconds, as observed via `watch kubectl get pods`. After stopping the requests for 60 seconds, the replicas scaled down to 0, which verified the auto-scaling-to-zero capability. When the service was updated to resident mode (`minScale=1`), 1 pod remained running continuously regardless of requests. The result is shown in Figure 2.

1.6.2 Knative Eventing Results

- **Broker and Service Creation:** The default Broker and event-display service were both created successfully, with "Ready" status displayed in the `kn broker list` and `kn service list` outputs. The result is shown in Figure 3.

- **Event Flow Verification:** The logs of the event-display service (obtained via `kubectl logs`) showed a log entry every 30 seconds, containing the "Hello from PingSource" message and event metadata such as type "dev.knative.sources.ping". This indicated that the PingSource successfully sent events to the Broker, and the Trigger correctly routed the filtered events to the event-display service. The result is shown in Figure 4.

1.6.3 Abnormal Situation Analysis

- **Webhook Component Image Pull Failure:** During the Knative deployment process, the webhook component failed to start, and the `kubectl describe pod <webhook-pod-name>` output showed an "ImagePullBackOff" error. The root cause was that the default image repository could not be accessed due to network restrictions.
- **Workaround:** Replace the default image with a domestic mirror. Edit the webhook deployment configuration via `kubectl edit deployment knative-serving-webhook -n knative-serving`, change the image address to a domestic accessible mirror (e.g., `registry.cn-hangzhou.aliyuncs.com/knative-serving/webhook:v1.10.0`), and save the configuration. After the pod restarted, the webhook component started successfully. This workaround was effective as it resolved the image pull issue by using a reachable mirror repository.
- **Event Display Service No Logs Received:** After creating the Trigger, no events were found in the event-display logs. The troubleshooting via `kn trigger describe event-trigger` revealed that the filter condition was incorrectly set to "type=dev.knative.ping" (missing "sources" in the type). After updating the Trigger with `kn trigger update event-trigger --filter type=dev.knative.sources.ping`, the service began receiving events normally. This issue was caused by incorrect parameter configuration, and correcting the filter condition resolved it.

1.7 Experiment Discussion

1.7.1 Advantages of Serverless Demonstrated by the Experiment

The experiment verified two core advantages of Serverless. First, **automatic resource scaling:** Knative Serving's auto-scaling mechanism dynamically adjusts the number of replicas based on request volume, avoiding resource waste caused by fixed replicas and ensuring service availability under high load. Scaling down to zero further optimizes resource utilization for services with intermittent traffic.

Second, **event-driven architecture:** Knative Eventing decouples event producers and consumers through Broker/Trigger. The PingSource (producer) does not need to know the address of the event-display service (consumer); the Broker acts as a

middle layer for event distribution, and Triggers implement fine-grained event filtering and routing, enhancing system flexibility and scalability.

1.7.2 Experiment Challenges and Solutions

The main challenge encountered was **incomplete Knative components due to network issues**. Some components (such as webhook and eventing-controller) failed to pull images, resulting in unavailable core functions. In addition to using domestic mirrors as a workaround, Kubernetes native functions were also used for simulation: for example, using HPA (Horizontal Pod Autoscaler) to simulate auto-scaling (setting scale based on CPU usage) and using Kubernetes Jobs to simulate periodic event sending. Although the simulation lacked some advanced features of Knative (such as scaling to zero), it verified the core logic and ensured the experiment proceeded smoothly.

1.7.3 Value of Knative in Microservices

Knative provides significant value in microservice architectures. For **deployment and management**, it simplifies the deployment of microservices by abstracting complex configurations such as auto-scaling and traffic routing into simple CLI commands, reducing operational costs. For **traffic management**, its fine-grained traffic splitting capability (not tested in this experiment) supports canary releases and blue-green deployments, reducing the risk of service updates. For **event-driven microservices**, the Broker/Trigger model provides a unified event bus, enabling seamless connection between microservices and avoiding tight coupling caused by direct calls. In scenarios such as e-commerce order processing (e.g., order creation triggering inventory updates and payment notifications), Knative can significantly improve system responsiveness and maintainability.

1.8 Conclusion

This experiment verified the core functions and features of Knative. Through the Knative Serving part, it mastered the basic operations of service creation and update, and confirmed the dual modes of Serverless applications (auto-scaling-to-zero and resident mode) and the effectiveness of the request-based auto-scaling mechanism. Through the Knative Eventing part, it understood the working principle of the Broker/Trigger-based event processing model and realized the complete event flow from event source to consumer. The solution to abnormal situations such as image pull failure improved the ability to troubleshoot Knative-related issues. Overall, this experiment achieved the expected objectives and laid a foundation for the practical application of Knative in Serverless and microservice scenarios.

1.9 Appendix: Experiment Screenshots

```
PS D:\Knative-Lab> # 安装Knative Eventing
PS D:\Knative-Lab> kubectl apply -f https://github.com/knative/eventing/releases/latest/download/eventing-crds.yaml
Warning: unrecognized format 'int64'
customresourcedefinition.apiextensions.k8s.io/apiserversources.sources.knative.dev created
Warning: unrecognized format 'int32'
customresourcedefinition.apiextensions.k8s.io/brokers.eventing.knative.dev created
customresourcedefinition.apiextensions.k8s.io/channels.messaging.knative.dev created
customresourcedefinition.apiextensions.k8s.io/containerresources.sources.knative.dev created
customresourcedefinition.apiextensions.k8s.io/eventpolicies.eventing.knative.dev created
customresourcedefinition.apiextensions.k8s.io/eventtransform.eventing.knative.dev created
customresourcedefinition.apiextensions.k8s.io/eventtypes.eventing.knative.dev created
customresourcedefinition.apiextensions.k8s.io/integrationsinks.sinks.knative.dev created
customresourcedefinition.apiextensions.k8s.io/integrationsources.sources.knative.dev created
customresourcedefinition.apiextensions.k8s.io/jobinks.sinks.knative.dev created
customresourcedefinition.apiextensions.k8s.io/parallelflows.knative.dev created
customresourcedefinition.apiextensions.k8s.io/pingsources.sources.knative.dev created
customresourcedefinition.apiextensions.k8s.io/requestreplies.eventing.knative.dev created
customresourcedefinition.apiextensions.k8s.io/sequences.flow.knative.dev created
customresourcedefinition.apiextensions.k8s.io/sinkbindings.sources.knative.dev created
customresourcedefinition.apiextensions.k8s.io/subscriptions.messaging.knative.dev created
customresourcedefinition.apiextensions.k8s.io/triggers.eventing.knative.dev created
PS D:\Knative-Lab> kubectl apply -f https://github.com/knative/serving/releases/latest/download/serving-crds.yaml
Warning: unrecognized format 'int64'
customresourcedefinition.apiextensions.k8s.io/certificates.networking.internal.knative.dev created
Warning: unrecognized format 'int32'
customresourcedefinition.apiextensions.k8s.io/configurations.serving.knative.dev created
customresourcedefinition.apiextensions.k8s.io/clusterdomainclaims.networking.internal.knative.dev created
customresourcedefinition.apiextensions.k8s.io/domainmappings.serving.knative.dev created
customresourcedefinition.apiextensions.k8s.io/ingresses.networking.internal.knative.dev created
customresourcedefinition.apiextensions.k8s.io/metrics.autoscaling.internal.knative.dev created
customresourcedefinition.apiextensions.k8s.io/podautoscalers.autoscaling.internal.knative.dev created
customresourcedefinition.apiextensions.k8s.io/revisions.serving.knative.dev created
customresourcedefinition.apiextensions.k8s.io/routes.serving.knative.dev created
customresourcedefinition.apiextensions.k8s.io/serverlessservices.networking.internal.knative.dev created
customresourcedefinition.apiextensions.k8s.io/services.serving.knative.dev created
customresourcedefinition.apiextensions.k8s.io/images.caching.internal.knative.dev created
PS D:\Knative-Lab> kubectl apply -f https://github.com/knative/serving/releases/latest/download/serving-core.yaml
namespace/knative-serving created
role.rbac.authorization.k8s.io/knative-serving-activator created
clusterrole.rbac.authorization.k8s.io/knative-serving-activator-cluster created
clusterrole.rbac.authorization.k8s.io/knative-serving-aggregated-addressable-resolver created
clusterrole.rbac.authorization.k8s.io/knative-serving-addressable-resolver created
clusterrole.rbac.authorization.k8s.io/knative-serving-namespaced-admin created
clusterrole.rbac.authorization.k8s.io/knative-serving-namespaced-edit created
clusterrole.rbac.authorization.k8s.io/knative-serving-namespaced-view created
clusterrole.rbac.authorization.k8s.io/knative-serving-core created
clusterrole.rbac.authorization.k8s.io/knative-serving-podspecable-binding created
serviceaccount/controller created
clusterrole.rbac.authorization.k8s.io/knative-serving-admin created
clusterrolebinding.rbac.authorization.k8s.io/knative-serving-controller-admin created
clusterrolebinding.rbac.authorization.k8s.io/knative-serving-controller-addressable-resolver created
serviceaccount/activator created
rolebinding.rbac.authorization.k8s.io/knative-serving-activator created
clusterrolebinding.rbac.authorization.k8s.io/knative-serving-activator-cluster created
customresourcedefinition.apiextensions.k8s.io/images.caching.internal.knative.dev unchanged
certificate.networking.internal.knative.dev/routing-serving-certs created
customresourcedefinition.apiextensions.k8s.io/certificates.networking.internal.knative.dev unchanged
customresourcedefinition.apiextensions.k8s.io/configurations.serving.knative.dev unchanged
customresourcedefinition.apiextensions.k8s.io/clusterdomainclaims.networking.internal.knative.dev unchanged
customresourcedefinition.apiextensions.k8s.io/domainmappings.serving.knative.dev unchanged
customresourcedefinition.apiextensions.k8s.io/ingresses.networking.internal.knative.dev unchanged
customresourcedefinition.apiextensions.k8s.io/metrics.autoscaling.internal.knative.dev unchanged

PS D:\Knative-Lab> # 创建Hello World服务配置
PS D:\Knative-Lab> cd D:\Knative-Lab\serving
PS D:\Knative-Lab\serving> # 创建Hello World服务配置
PS D:\Knative-Lab\serving> kubectl apply -f .
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: hello
spec:
  template:
    spec:
      containers:
        - image: ghcr.io/knative/helloworld-go:latest
          ports:
            - containerPort: 8080
          env:
            - name: TARGET
              value: "World"
            - name: "Out-File"
              value: "filePath 'hello.yaml' -encoding UTF8"
            - name: "Out-File"
              value: "filePath 'hello.yaml' -encoding UTF8"
PS D:\Knative-Lab\serving> # 查看创建的文件
PS D:\Knative-Lab\serving> cat hello.yaml
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: hello
spec:
  template:
    spec:
      containers:
        - image: ghcr.io/knative/helloworld-go:latest
          ports:
            - containerPort: 8080
          env:
            - name: TARGET
              value: "World"
```

```
PS D:\Knative-Lab\serving> kubectl get pods -A | findstr -i running
default      bookinfo-gateway-istio-7db9c6d6ff-lcr78      1/1      Running      3 (3m15s ago)  26d
default      details-v1-77d6bd5675-jzfnx                 2/2      Running      2 (5m5s ago)   26d
default      hello-node-6c9b5f4b59-trl74                 1/1      Running      3 (3m20s ago)  26d
default      productpage-v1-bb87ff47b-r7mm2              2/2      Running      2 (4m56s ago)  26d
default      ratings-v1-8589f64b4c-fc2qs                 2/2      Running      2 (5m4s ago)   26d
default      reviews-v1-8cf7b9cc5-nmj5v                 2/2      Running      2 (4m56s ago)  26d
default      reviews-v2-67d565655f-mjqfp                2/2      Running      2 (4m56s ago)  26d
default      reviews-v3-d587fc9d7-pd68x                 2/2      Running      2 (4m56s ago)  26d
istio-system istio-egressgateway-69b8465445-bwdtf         1/1      Running      3 (3m16s ago)  26d
istio-system istio-ingressgateway-7554995647-b8sjx        1/1      Running      3 (3m15s ago)  26d
istio-system istiod-d468976dc-7b5wk                      1/1      Running      3 (3m10s ago)  26d
kube-system  coredns-7ddb67b59b-8mrcp                    1/1      Running      3 (3m15s ago)  26d
kube-system  coredns-7ddb67b59b-ncknv                    1/1      Running      3 (3m15s ago)  26d
kube-system  etcd-minikube                               1/1      Running      3 (3m13s ago)  26d
kube-system  kube-apiserver-minikube                     1/1      Running      3 (3m10s ago)  26d
kube-system  kube-controller-manager-minikube            1/1      Running      4 (3m20s ago)  26d
kube-system  kube-proxy-bxc76                            1/1      Running      3 (3m20s ago)  26d
kube-system  kube-scheduler-minikube                     1/1      Running      3 (3m20s ago)  26d
kube-system  storage-provisioner                         1/1      Running      5 (2m54s ago)  26d
kubernetes-dashboard dashboard-metrics-scraper-77bf4d6c4c-fmr28 1/1      Running      3 (3m20s ago)  26d
kubernetes-dashboard kubernetes-dashboard-855c9754f9-gv16c      1/1      Running      6 (2m47s ago)  26d
PS D:\Knative-Lab\serving> kubectl get crd | Select-String -Pattern "knative"
```

```
apiserversources.sources.knative.dev          2025-12-15T07:23:42Z
brokers.eventing.knative.dev                  2025-12-15T07:23:42Z
certificates.networking.internal.knative.dev  2025-12-15T07:25:04Z
channels.messaging.knative.dev                2025-12-15T07:23:42Z
clusterdomainclaims.networking.internal.knative.dev
configurations.serving.knative.dev            2025-12-15T07:25:04Z
containersources.sources.knative.dev          2025-12-15T07:23:42Z
domainmappings.serving.knative.dev            2025-12-15T07:25:05Z
eventpolicies.eventing.knative.dev            2025-12-15T07:23:42Z
eventtransforms.eventing.knative.dev          2025-12-15T07:23:42Z
eventtypes.eventing.knative.dev               2025-12-15T07:23:42Z
images.caching.internal.knative.dev           2025-12-15T07:25:05Z
ingresses.networking.internal.knative.dev     2025-12-15T07:25:05Z
integrationsinks.sinks.knative.dev            2025-12-15T07:23:43Z
integrationsources.sources.knative.dev        2025-12-15T07:23:43Z
jobsinks.sinks.knative.dev                   2025-12-15T07:23:43Z
metrics.autoscaling.internal.knative.dev      2025-12-15T07:25:05Z
parallels.flows.knative.dev                   2025-12-15T07:23:43Z
pingsources.sources.knative.dev               2025-12-15T07:23:43Z
podautoscalers.autoscaling.internal.knative.dev
requestreplies.eventing.knative.dev           2025-12-15T07:23:43Z
revisions.serving.knative.dev                 2025-12-15T07:25:05Z
routes.serving.knative.dev                    2025-12-15T07:25:05Z
sequences.flows.knative.dev                   2025-12-15T07:23:43Z
serverlesservices.networking.internal.knative.dev
services.serving.knative.dev                  2025-12-15T07:25:05Z
sinkbindings.sources.knative.dev              2025-12-15T07:23:43Z
subscriptions.messaging.knative.dev           2025-12-15T07:23:43Z
triggers.eventing.knative.dev                 2025-12-15T07:23:43Z
```

```
PS D:\Knative-Lab\serving> cd D:\Knative-Lab\serving
PS D:\Knative-Lab\serving> @
>> apiVersion: serving.knative.dev/v1
>> kind: Service
>> metadata:
>>   name: knative-hello
>> spec:
>>   template:
>>     spec:
>>       containers:
>>         - image: nginx:alpine
>>         ports:
>>           - containerPort: 80
>> "@ | Out-File -FilePath "knative-hello-attempt.yaml" -Encoding UTF8
PS D:\Knative-Lab\serving> cat knative-hello-attempt.yaml
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: knative-hello
spec:
  template:
    spec:
      containers:
        - image: nginx:alpine
          ports:
            - containerPort: 80
```

```
PS D:\Knative-Lab\serving>
PS D:\Knative-Lab\serving> cat knative-hello-attempt.yaml
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: knative-hello
spec:
  template:
    spec:
      containers:
        - image: nginx:alpine
          ports:
            - containerPort: 80
PS D:\Knative-Lab\serving> kubectl apply -f knative-hello-attempt.yaml
Error from server (InternalError): error when creating "knative-hello-attempt.yaml": Internal error occurred: failed calling webhook "webhook.serving.knative.dev": failed to call webhook: Post "https://webhook.knative-serving.svc:443/?timeout=10s": dial tcp 10.109.251.192:443: connect: connection refused
PS D:\Knative-Lab\serving> kubectl get crd | Select-String -Pattern "serving.knative.dev"
configurations.serving.knative.dev          2025-12-15T07:25:04Z
domainmappings.serving.knative.dev          2025-12-15T07:25:05Z
revisions.serving.knative.dev               2025-12-15T07:25:05Z
routes.serving.knative.dev                  2025-12-15T07:25:05Z
services.serving.knative.dev                2025-12-15T07:25:05Z
PS D:\Knative-Lab\serving> kubectl get crd | Select-String -Pattern "eventing.knative.dev"
brokers.eventing.knative.dev                2025-12-15T07:23:42Z
eventpolicies.eventing.knative.dev          2025-12-15T07:23:42Z
eventtransforms.eventing.knative.dev        2025-12-15T07:23:42Z
eventtypes.eventing.knative.dev             2025-12-15T07:23:42Z
requestreplies.eventing.knative.dev         2025-12-15T07:23:43Z
triggers.eventing.knative.dev               2025-12-15T07:23:43Z
```

```

PS D:\Knative-Lab\-serving> # 查看当前状态
PS D:\Knative-Lab\-serving> kubectl get deployment hello-node
NAME READY UP-TO-DATE AVAILABLE AGE
hello-node 1/1 1 1 26d
PS D:\Knative-Lab\-serving> kubectl get pods -l app=hello-node
NAME READY STATUS RESTARTS AGE
hello-node-6c9b5f4b59-tr174 1/1 Running 3 (8m39s ago) 26d
PS D:\Knative-Lab\-serving>
PS D:\Knative-Lab\-serving> # 将副本数缩容到0, 模拟无请求时的状态
PS D:\Knative-Lab\-serving> kubectl scale deployment hello-node --replicas=0
deployment.apps/hello-node scaled
PS D:\Knative-Lab\-serving>
PS D:\Knative-Lab\-serving> # 观察Pod全部终止
PS D:\Knative-Lab\-serving> kubectl get pods -l app=hello-node
NAME READY STATUS RESTARTS AGE
hello-node-6c9b5f4b59-tr174 1/1 Terminating 3 (8m40s ago) 26d
PS D:\Knative-Lab\-serving>
PS D:\Knative-Lab\-serving> # 将副本数恢复为1, 模拟请求到达
PS D:\Knative-Lab\-serving> kubectl scale deployment hello-node --replicas=1
deployment.apps/hello-node scaled
PS D:\Knative-Lab\-serving>
PS D:\Knative-Lab\-serving> # 观察Pod重新创建并变为Ready
PS D:\Knative-Lab\-serving> kubectl get pods -l app=hello-node -w
NAME READY STATUS RESTARTS AGE
hello-node-6c9b5f4b59-4hbkf 0/2 Init:0/2 0 0s
hello-node-6c9b5f4b59-tr174 0/1 Completed 3 (8m40s ago) 26d
hello-node-6c9b5f4b59-tr174 0/1 Completed 3 (8m41s ago) 26d
hello-node-6c9b5f4b59-tr174 0/1 Completed 3 (8m41s ago) 26d
hello-node-6c9b5f4b59-4hbkf 0/2 Init:1/2 0 2s
hello-node-6c9b5f4b59-4hbkf 0/2 Init:1/2 0 3s
hello-node-6c9b5f4b59-4hbkf 0/2 PodInitializing 0 3s
hello-node-6c9b5f4b59-4hbkf 0/2 PodInitializing 0 4s
hello-node-6c9b5f4b59-4hbkf 1/2 PodInitializing 0 4s
hello-node-6c9b5f4b59-4hbkf 2/2 Running 0 5s
PS D:\Knative-Lab\-serving> # 为 hello-node 创建 HorizontalPodAutoscaler, 模拟基于CPU指标的自动伸缩
PS D:\Knative-Lab\-serving> kubectl autoscale deployment hello-node --cpu-percent=50 --min=1 --max=3
horizontalpodautoscaler.autoscaling/hello-node autoscaled
PS D:\Knative-Lab\-serving>
PS D:\Knative-Lab\-serving> # 查看创建的HPA
PS D:\Knative-Lab\-serving> kubectl get hpa hello-node
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
hello-node Deployment/hello-node cpu: <unknown>/50% 1 3 0 0s
PS D:\Knative-Lab\-serving> kubectl describe hpa hello-node
S D:\Knative-Lab\-serving> kubectl describe hpa hello-nodecd D:\Knative-Lab\eventing
error from server (NotFound): horizontalpodautoscalers.autoscaling "hello-nodecd" not found
error from server (NotFound): horizontalpodautoscalers.autoscaling "D:\\Knative-Lab\\eventing" not found
PS D:\Knative-Lab\-serving> @
> apiVersion: batch/v1
> kind: CronJob
> metadata:
>   name: event-producer
> spec:
>   schedule: "* * * * *" # 每分钟触发一次
>   jobTemplate:
>     spec:
>       template:
>         spec:
>           containers:
>             - name: producer
>               image: busybox
>               command:
>               - /bin/sh
>               - -c
>               - 'echo `date`: [EVENT] Ping from CronJob. Sending simulated payload.'
>               # 在实际场景中, 这里会向一个消息队列或HTTP端点发送事件
>             restartPolicy: OnFailure
> "@ | kubectl apply -f -
cronjob.batch/event-producer created
PS D:\Knative-Lab\-serving>

```

```
PS D:\Knative-Lab\event-producer> kubectl get cronjobs.batch
NAME          SCHEDULE    TIMEZONE  SUSPEND  ACTIVE  LAST SCHEDULE  AGE
event-producer  * * * * *  <none>    False    0        <none>         23s
PS D:\Knative-Lab\event-producer> kubectl get jobs.batch --watch
NAME          STATUS    COMPLETIONS  DURATION  AGE
event-producer-29429753  Running  0/1           0s        0s
event-producer-29429753  Running  0/1           0s        0s
event-producer-29429754  Running  0/1           0s        0s
event-producer-29429754  Running  0/1           0s        0s
PS D:\Knative-Lab\event-producer> kubectl get pods --selector=job-name=<这里替换为实际的job名称> -o jsonpath='{.items[*].metadata.name}'
Error from server (BadRequest): Unable to find "/v1. Resource=pods" that match label selector "job-name=<这里替换为实际的job名称>", field selector ": unable to parse requirement: found '<', expected: identifier"
PS D:\Knative-Lab\event-producer> # 使用上一步得到的Pod名称查看日志
PS D:\Knative-Lab\event-producer> kubectl logs event-producer-29429753 -o jsonpath='{.items[0].metadata.name}'
event-producer-29429753
error: error from server (NotFound): pods "event-producer-29429753" not found in namespace "default"
PS D:\Knative-Lab\event-producer> echo 事件生产者 Pod 名称: $JOB_NAME
事件生产者 Pod 名称: event-producer-29429753-6v4rs
PS D:\Knative-Lab\event-producer> # 查看 Pod 的日志输出
PS D:\Knative-Lab\event-producer> kubectl logs $JOB_NAME
Error from server (BadRequest): container "producer" in pod "event-producer-29429753-6v4rs" is waiting to start: trying and failing to pull image
PS D:\Knative-Lab\event-producer> # 确保在正确的目录
PS D:\Knative-Lab\event-producer> cd D:\Knative-Lab\eventing
PS D:\Knative-Lab\eventing> # 创建正确的CronJob YAML文件
PS D:\Knative-Lab\eventing> cat <这里替换为实际的job名称>.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: event-producer
spec:
  schedule: "* * * * *" # 每分钟触发一次
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: producer
              image: busybox
              command:
                - /bin/sh
                - -c
                - 'echo "date: [EVENT] Ping from CronJob. Sending simulated payload."'
          restartPolicy: OnFailure
  > kubectl apply -f -
cronjob.batch/event-producer unchanged
PS D:\Knative-Lab\eventing> # 查看CronJob创建
PS D:\Knative-Lab\eventing> kubectl get cronjobs.batch event-producer
NAME          SCHEDULE    TIMEZONE  SUSPEND  ACTIVE  LAST SCHEDULE  AGE
event-producer  * * * * *  <none>    False    6        27s         6m19s
PS D:\Knative-Lab\eventing> # 等待2分钟让CronJob触发
PS D:\Knative-Lab\eventing> Write-Host "等待CronJob触发 (约60秒) ..." -ForegroundColor Yellow
等待CronJob触发 (约60秒) ...
PS D:\Knative-Lab\eventing> Start-Sleep -Seconds 60
PS D:\Knative-Lab\eventing> # 查看创建的jobs
PS D:\Knative-Lab\eventing> kubectl get jobs
NAME          STATUS    COMPLETIONS  DURATION  AGE
event-producer-29429753  Running  0/1           7m        7m
event-producer-29429754  Running  0/1           6m        6m
event-producer-29429755  Running  0/1           5m        5m
event-producer-29429756  Running  0/1           4m        4m
event-producer-29429757  Running  0/1           3m        3m
event-producer-29429758  Running  0/1           2m        2m
event-producer-29429759  Running  0/1           60s       60s
event-producer-29429760  Running  0/1           0s        0s
PS D:\Knative-Lab\eventing> # 找到最新的job (名称以event-producer开头)
PS D:\Knative-Lab\eventing> $JOB_NAME = kubectl get jobs -o jsonpath='{.items[?(.metadata.name -match "event-producer-").metadata.name]} | Select-Object -First 1'
error: error parsing jsonpath '{.items[?(.metadata.name -match "event-producer-").metadata.name]} | Select-Object -First 1': cannot parse number -
PS D:\Knative-Lab\eventing> Write-Host "找到Job: $JOB_NAME"
找到Job:
PS D:\Knative-Lab\eventing> # 获取这个job创建的pod
PS D:\Knative-Lab\eventing> $POD_NAME = kubectl get pods --selector=job-name=$JOB_NAME -o jsonpath='{.items[0].metadata.name}'
error: error executing jsonpath "{.items[0].metadata.name}": Error executing template: array index out of bounds: index 0, length 0. Printing more information for debugging the template:
  template was:
    {.items[0].metadata.name}
  object given to jsonpath engine was:
    map[string]interface {} {"apiVersion":"v1", "items":[]interface {} [], "kind":"List", "metadata":map[string]interface {} {"resourceVersion":""}}
PS D:\Knative-Lab\eventing> Write-Host "找到Pod: $POD_NAME"
找到Pod:
PS D:\Knative-Lab\eventing> # 查看Pod日志
PS D:\Knative-Lab\eventing> kubectl logs $POD_NAME
error: expected 'logs [-f] [-p] (POD | TYPE/NAME) [-c CONTAINER]'.
POD or TYPE/NAME is a required argument for the logs command
See 'kubectl logs -h' for help and examples
PS D:\Knative-Lab\eventing> # 再次查看HPA状态
PS D:\Knative-Lab\eventing> kubectl get hpa hello-node
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
hello-node    Deployment/hello-node  cpu: unknown/50%  1          3          1          8m37s
PS D:\Knative-Lab\eventing> kubectl describe hpa hello-node
Name:          hello-node
Namespace:     default
Labels:        <none>
Annotations:   <none>
CreationTimestamp: Mon, 15 Dec 2025 15:51:48 +0800
Reference:     Deployment/hello-node
Metrics:       ( current / target )
  resource cpu on pods (as a percentage of request):  unknown / 50%
Min replicas:  1
Max replicas:  3
Deployment pods:  1 current / 0 desired
Conditions:
  Type           Status  Reason                        Message
  --           --
  AbleToScale    True    SucceededGetScale              the HPA controller was able to get the target's current scale
```

[illegible]